AI Pull Request Reviewer: Database Documentation

Nov 6, 2026 · @Aashu

The database stores the Repository Knowledge Base (RKB), a structured map of files, symbols and dependencies that the AI reviewer uses to assess the impact of Pull Request changes.

1. Overview

The system uses one shared PostgreSQL database (hosted on Supabase, accessed through SQLAlchemy, migrated with Alembic) for many users and many GitHub repositories.

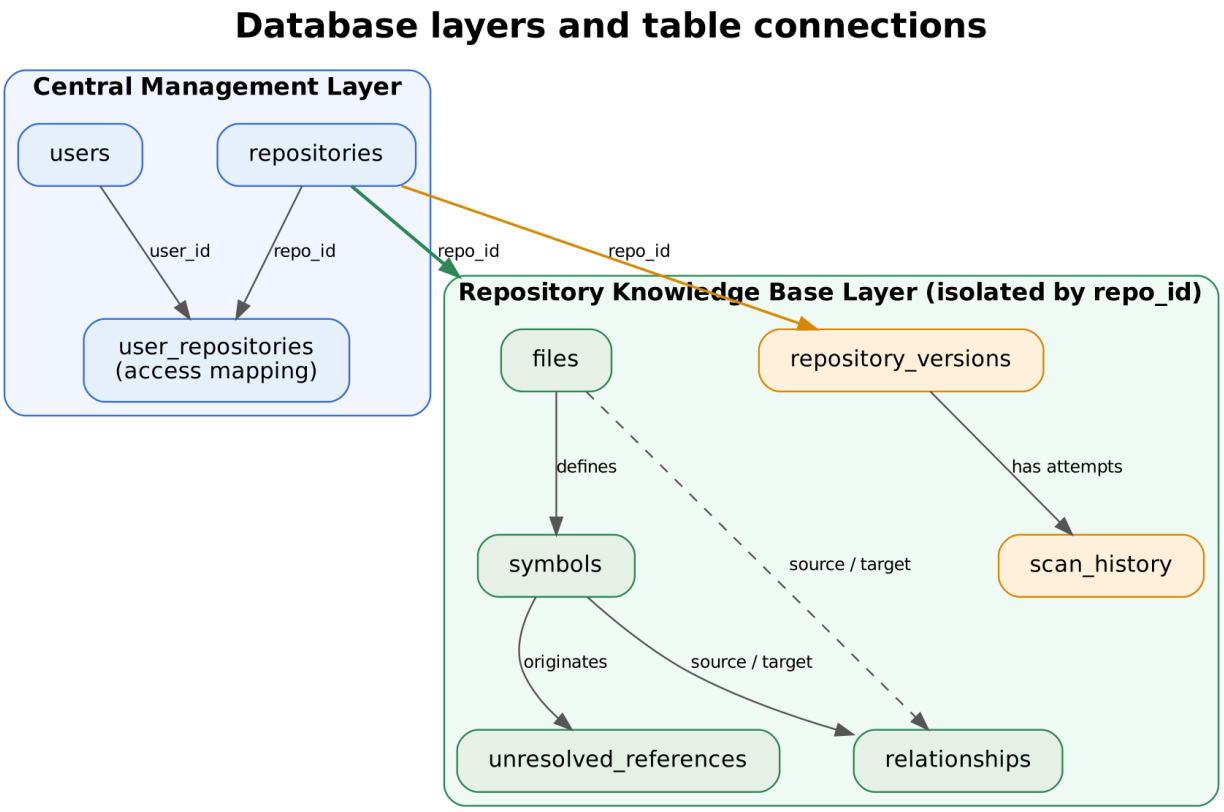
- **Isolation:** every repository gets its own internal `repo_id`, and all knowledge-base rows carry it, so repositories never mix.
- **Sharing:** users who connect the same GitHub repository reuse one repository record and one knowledge base.
- **Ownership:** knowledge-base data belongs to the repository, not to a user. Access is controlled only through `user_repositories`, so `user_id` is not duplicated across knowledge-base rows.

2. Architecture

The database has two logical layers. Both live in the same PostgreSQL instance.

Layer	Tables	Responsibility
Central management	`users`, `repositories`, `user_repositories`	Identities, connected GitHub repositories, and who may access which repository
Repository Knowledge Base (RKB)	`files`, `symbols`, `relationships`, `unresolved_references`	Code structure extracted from source files
Scan tracking (part of RKB)	`repository_versions`, `scan_history`	Which commits were scanned, their status, and every scan attempt

Example: User A connects Repositories X and Y; User B connects X. Both users share X's knowledge base, and Y keeps a separate one.



3. Central management tables

These tables record who the users are, which repositories exist, and which users may access them. Primary keys are UUIDs generated by `gen_random_uuid()`. Audit timestamps (`created_at`, `updated_at`) are omitted from the tables below.

3.1 users

One row per application user, created on first GitHub sign-in. An existing `github_id` reuses the existing `user_id`.

Column	Type	Constraint	Description
`user_id`	UUID	Primary key	Internal user identifier
`github_id`	BIGINT	Unique, not null	GitHub account ID
`github_username`	VARCHAR(255)	Not null	GitHub username
`email`	VARCHAR(255)	Nullable	Email address

Column	Type	Constraint	Description
`avatar_url`	TEXT	Nullable	GitHub profile picture URL

3.2 repositories

One row per connected GitHub repository. The application looks up `github_repo_id` first and reuses the existing `repo_id` when a repository is already registered.

Column	Type	Constraint	Description
`repo_id`	UUID	Primary key	Internal repository identifier, used across the RKB
`github_repo_id`	BIGINT	Unique, not null	GitHub repository ID
`repo_name`	VARCHAR(255)	Not null	Repository name
`full_name`	VARCHAR(500)	Not null	Owner and name
`description`	TEXT	Nullable	Repository description
`default_branch`	VARCHAR(255)	Nullable	Default branch
`visibility`	VARCHAR(20)	Nullable	Public or private
`clone_url`	TEXT	Not null	Git clone URL

3.3 user_repositories

Join table for the many-to-many link between users and repositories. The backend uses it to authorize every repository request.

Column	Type	Constraint	Description
`user_id`	UUID	Composite PK, FK to `users`	The user
`repo_id`	UUID	Composite PK, FK to `repositories`	The repository

Both foreign keys use `ON DELETE CASCADE`, so removing a user or repository removes its mappings.

4. Repository Knowledge Base tables

These tables hold the structure extracted from source code. Every row carries `repo_id`, and composite foreign keys `(repo_id, ...)` guarantee that linked rows belong to the same repository. Source code itself is not stored, only metadata and structure.

4.1 files

One row per source file found during a scan.

Column	Type	Description
`file_id`	UUID (PK)	File identifier
`repo_id`	UUID (FK)	Owning repository
`file_path`	TEXT	Path relative to the repository root
`file_name`, `extension`	VARCHAR	Name and extension
`language`	VARCHAR(100)	Detected language
`content_hash`	VARCHAR(64)	Hash used to detect changes
`size_bytes`	BIGINT	File size
`is_deleted`	BOOLEAN	Marks files removed from the repository

Unique on `(repo_id, file_path)`, so two repositories can each have `src/main.py`. Also unique on `(repo_id, file_id)` to support composite foreign keys.

4.2 symbols

Code entities (classes, functions, methods, interfaces, constants) extracted with Tree-sitter, each tied to the file that declares it.

Column	Type	Description
`symbol_id`	UUID (PK)	Symbol identifier
`repo_id`, `file_id`	UUID (FK)	Owning repository and declaring file, as a composite key
`parent_symbol_id`	UUID (FK, nullable)	Enclosing symbol, for example the class of a method
`symbol_name`, `qualified_name`	VARCHAR / TEXT	Short and fully qualified names

Column	Type	Description
`symbol_type`	VARCHAR(50)	Class, function, method and so on
`signature`	TEXT	Signature, where applicable
`start_line`, `end_line`	INTEGER	Location in the file
`visibility`, `docstring`	VARCHAR / TEXT	Access level and documentation

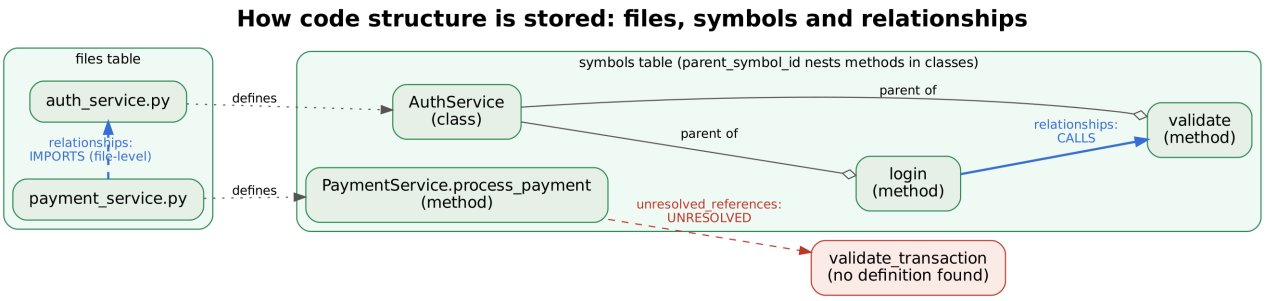
4.3 relationships

The dependency graph. Each row is either a file-to-file or a symbol-to-symbol link, enforced by the `valid_relationship_endpoints` check constraint. Mixed file-to-symbol links are not supported.

Column	Type	Description
`relationship_id`	UUID (PK)	Relationship identifier
`repo_id`	UUID (FK)	Owning repository
`source_file_id`, `target_file_id`	UUID (FK, nullable)	Endpoints of a file-level link
`source_symbol_id`, `target_symbol_id`	UUID (FK, nullable)	Endpoints of a symbol-level link
`relationship_type`	VARCHAR(50)	Kind of dependency

Example types, not enforced by a database constraint:

Type	Meaning
`IMPORTS`	A file imports another file or module
`CALLS`	A function or method calls another
`INHERITS`	A class inherits from another
`IMPLEMENTS`	A class implements an interface
`USES`	An entity uses another entity



The diagram shows an import between two files, a method call between two symbols, and a call to a function with no known definition.

4.4 repository_versions

One row per scanned commit. Only one version per repository can be active, and only a `COMPLETED` version can be active.

Column	Type	Description
`version_id`	UUID (PK)	Version identifier
`repo_id`	UUID (FK)	Owning repository
`commit_sha`	VARCHAR(40)	Scanned commit
`branch_name`	VARCHAR(255)	Branch, if known
`scan_status`	VARCHAR(30)	`PENDING`, `IN_PROGRESS`, `COMPLETED` or `FAILED`
`is_active`	BOOLEAN	Whether this is the version currently in use
`started_at`, `completed_at`	TIMESTAMPTZ	Scan timing

4.5 scan_history

An audit log with one row per scan attempt, including failures. `version_id` is nullable so an attempt can be logged even when no version was created, and it is set to NULL if the version is deleted.

Column	Type	Description
`scan_id`	UUID (PK)	Scan identifier
`repo_id`	UUID (FK)	Scanned repository

Column	Type	Description
`version_id`	UUID (FK, nullable)	Related version
`commit_sha`, `branch_name`	VARCHAR	Target commit and branch
`scan_type`, `scan_status`	VARCHAR(30)	Kind of scan and its outcome
`error_message`	TEXT	Failure details
`started_at`, `completed_at`	TIMESTAMPTZ	Scan timing

4.6 unresolved_references

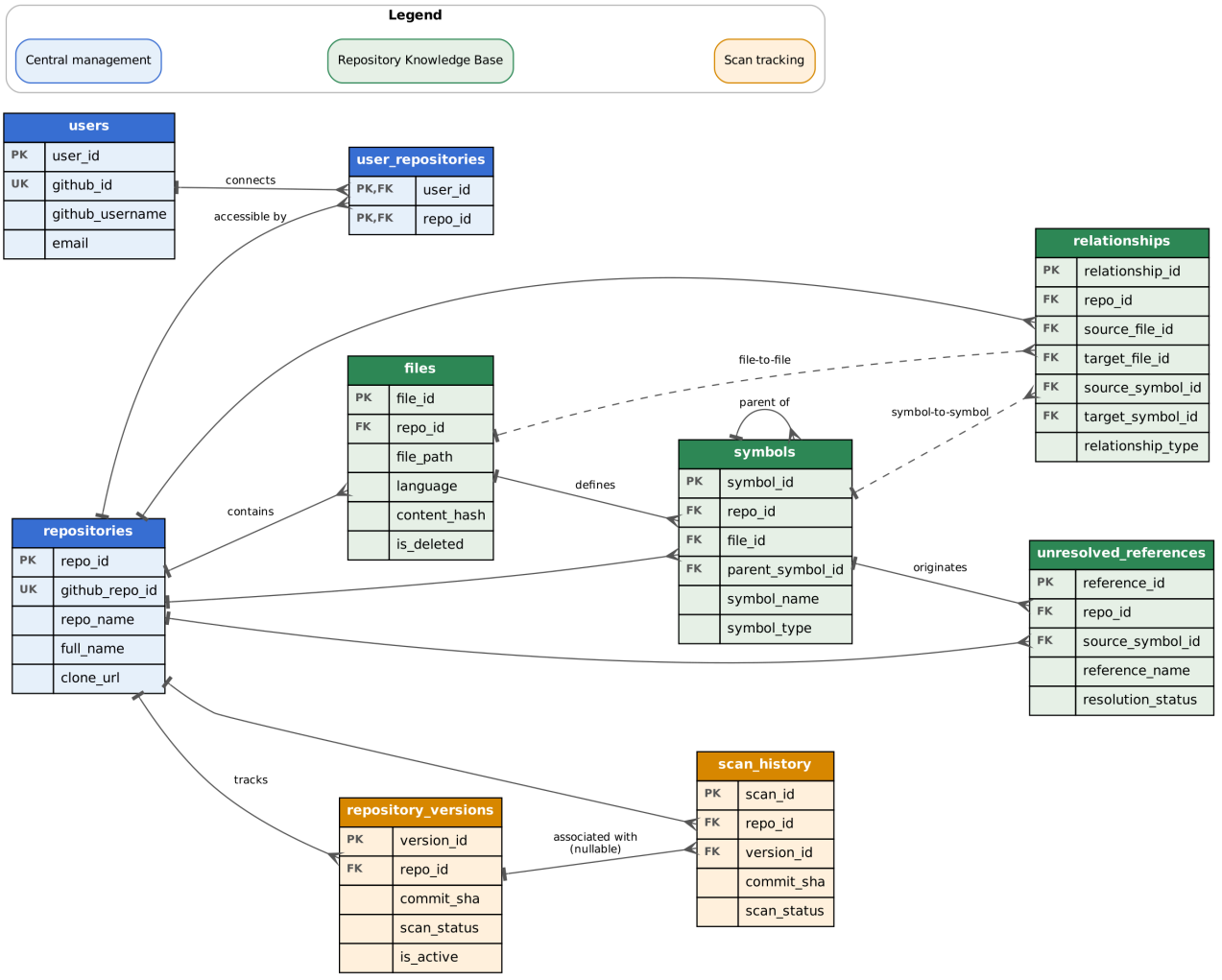
References whose target could not be matched to a known symbol. They are kept rather than discarded, which helps debug the resolver and flags missing or external dependencies.

Column	Type	Description
`reference_id`	UUID (PK)	Reference identifier
`repo_id`, `source_symbol_id`	UUID (FK)	Referencing symbol, as a composite key
`reference_name`	VARCHAR(255)	Name that could not be resolved
`reference_type`	VARCHAR(50)	For example function call or class reference
`line_number`	INTEGER	Line of the reference
`resolution_status`	VARCHAR(30)	`UNRESOLVED` or `AMBIGUOUS` (multiple candidates)

5. Entity relationship diagram

Blue tables are central management, green tables are the knowledge base, and orange tables track scans. Dashed lines on `relationships` show that each row uses either the file pair or the symbol pair.

Entity relationship diagram



From	To	Cardinality	Meaning
`users`	`user_repositories`	One to many	A user can connect many repositories
`repositories`	`user_repositories`	One to many	A repository can be connected by many users
`repositories`	`files`, `symbols`, `relationships`, `unresolved_references`, `repository_versions`, `scan_history`	One to many	Each knowledge-base row belongs to one repository
`files`	`symbols`	One to many	A file declares many symbols

From	To	Cardinality	Meaning
`symbols`	`symbols`	Self-reference	A symbol can contain child symbols
`files` or `symbols`	`relationships`	One to many	Endpoints of dependency links
`symbols`	`unresolved_references`	One to many	A symbol can hold many unresolved references
`repository_versions`	`scan_history`	One to many, optional	A version can have many scan attempts

6. Constraints, indexes and cascades

All main tables use UUID primary keys generated by `gen_random_uuid()`; `user_repositories` uses the composite key `(user_id, repo_id)`. Composite foreign keys keep related rows inside the same repository.

6.1 Unique constraints

Table	Columns	Purpose
`users`	`github_id`	No duplicate GitHub accounts
`repositories`	`github_repo_id`	No duplicate repositories
`files`	`(repo_id, file_path)`	No duplicate paths within a repository
`files`	`(repo_id, file_id)`	Target for composite foreign keys
`symbols`	`(repo_id, symbol_id)`	Target for composite foreign keys
`repository_versions`	`(repo_id, version_id)`	Target for composite foreign keys

6.2 Check constraints

Table	Rule
`repository_versions`	`scan_status` is `PENDING`, `IN_PROGRESS`, `COMPLETED` or `FAILED`; an active version must be `COMPLETED`
`relationships`	Each row is file-to-file or symbol-to-symbol, never mixed
`unresolved_references`	`resolution_status` is `UNRESOLVED` or `AMBIGUOUS`

A partial unique index, `one_active_version_per_repo`, allows only one active version per repository.

6.3 Indexes

Index	Table	Columns	Supports
`idx_symbols_repo_file`	`symbols`	`(repo_id, file_id)`	Symbols of a file
`idx_symbols_repo_parent`	`symbols`	`(repo_id, parent_symbol_id)`	Child symbols
`idx_rel_source_file`	`relationships`	`(repo_id, source_file_id)`	Links leaving a file
`idx_rel_target_file`	`relationships`	`(repo_id, target_file_id)`	Links entering a file
`idx_rel_source_symbol`	`relationships`	`(repo_id, source_symbol_id)`	Links leaving a symbol
`idx_rel_target_symbol`	`relationships`	`(repo_id, target_symbol_id)`	Links entering a symbol
`idx_versions_repo_created`	`repository_versions`	`(repo_id, created_at DESC)`	Latest versions first
`idx_scan_history_repo_created`	`scan_history`	`(repo_id, started_at DESC)`	Recent scans first
`idx_unresolved_repo_symbol`	`unresolved_references`	`(repo_id, source_symbol_id)`	References of a symbol

6.4 Cascade behavior

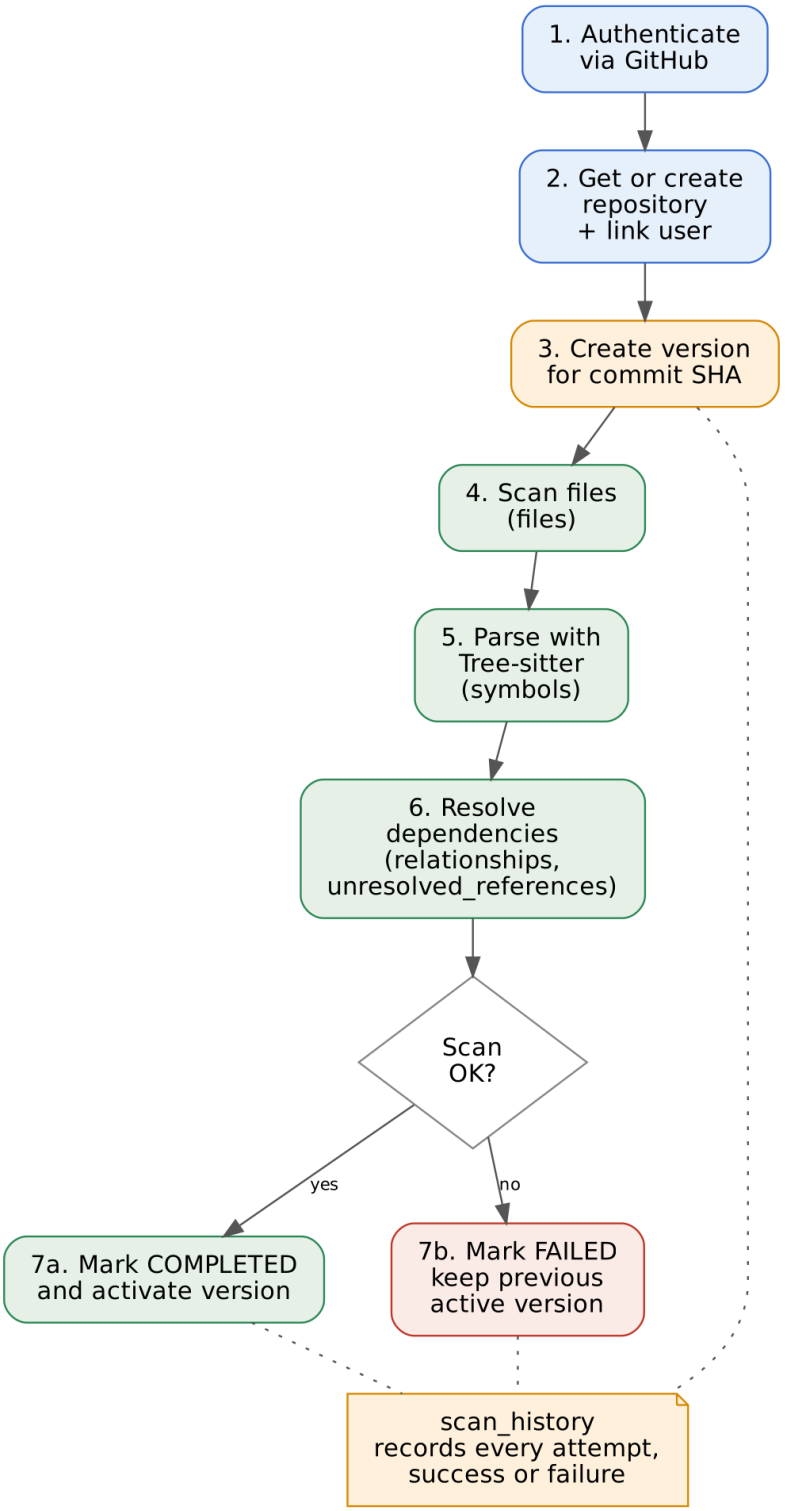
- Deleting a repository removes its knowledge-base records.

- Deleting a file removes its symbols; deleting a symbol removes its child symbols and unresolved references.
- Deleting a referenced file or symbol removes the relationships that point to it.
- Deleting a user removes that user's repository mappings.
- Deleting a repository version sets `scan_history.version_id` to NULL and keeps the history row.

7. Scanning workflow and version lifecycle

A scan runs the steps below. A new version becomes active only after the whole scan succeeds, so a failed scan never replaces the last good state.

Repository scan lifecycle and the tables each step writes



1. Authenticate the user through GitHub.
2. Identify the repository by its GitHub repository ID.
3. Get or create the repository record and link the user to it.
4. Resolve the commit SHA to scan and create a repository version.
5. Scan repository files and store their metadata.

6. Parse supported files with Tree-sitter and extract symbols.
7. Resolve dependencies, store relationships, and record unresolved references.
8. Mark the scan `COMPLETED` and activate the version, or mark it `FAILED` and keep the previous active version.

Every step receives a shared scan context so all components work on the same repository and scan:

Field	Identifies
`user_id`	The authenticated user who started the scan
`repo_id`	The repository being scanned
`commit_sha`	The exact commit being analyzed
`version_id`	The repository version created for this scan

The backend must take the user's identity from the authenticated session and never trust a `user_id` sent by the frontend.

8. Validation

The schema was validated with sample records inserted directly into the Supabase PostgreSQL database. All checks passed, and the sample data was then removed with `TRUNCATE ... RESTART IDENTITY`, leaving the schema, keys, indexes and constraints intact.

Area	What was checked	Result
Multi-user access	Two users linked to one repository share a single `repo_id` and knowledge base	Passed
Multi-repository isolation	Two repositories store the same file path without conflict	Passed
Symbol storage	Symbols link to valid files; parent symbols stay in the same repository	Passed
Relationship integrity	`CALLS` links between same-repository symbols stored; invalid endpoints rejected	Passed
Repository versions	Only one active version; active requires `COMPLETED`	Passed

Area	What was checked	Result
Scan history	Successful and failed scans logged; deleting a version nulls `version_id`	Passed
Unresolved references	`UNRESOLVED` and `AMBIGUOUS` stored; same-repository symbol enforced	Passed

9. Known limitations and planned improvements

The schema isolates data by repository but is not yet ready for safe staged scans. The first row is the most important gap.

Area	Current status	Planned improvement
Version-level isolation	`files`, `symbols`, `relationships` and `unresolved_references` have no `version_id`, so scans of one repository cannot be isolated from each other	Add `version_id` to these tables with composite foreign keys on `(repo_id, version_id)`
Staging and atomic activation	`is_active` and the partial unique index exist, but no staged workflow	Populate a new version separately, then activate it and deactivate the old one in one transaction
Incremental scanning	`content_hash` is stored but not used	Classify files as added, modified, unchanged or deleted, and reprocess only affected files and dependents
Relationship model	Only file-to-file and symbol-to-symbol links	Allow mixed links, such as a test function referencing production code, and more link types
Legacy table	`repository_database_registry` is left from the per-repository database design	Remove it after final migration and dependency checks
Schema migrations	Schema was created in the Supabase SQL Editor	Manage changes with SQLAlchemy and Alembic migrations

Area	Current status	Planned improvement
Access control	`user_repositories` exists, but authorization and database policies are not implemented	Check access before every repository operation, derive identity from the session, keep privileged credentials on the backend, and add Row Level Security if clients access the database directly
Optimization	Indexes are based on expected queries	Tune indexes from real query patterns, add monitoring, scan retention policies, large-repository handling and richer scan metrics